

Top 20 SQL Questions (with answers) — For Data Engineers



Here are 20 FAQ SQL questions you'll commonly face as a data engineer.

1) What's the difference between INNER JOIN, LEFT JOIN, RIGHT JOIN, and FULL OUTER JOIN?

Answer:

- INNER JOIN: returns rows with matching keys in both tables.
- LEFT JOIN: returns all rows from left table + matching rows from right (NULL if no match).
- RIGHT JOIN: all rows from right table + matches from left.
- FULL OUTER JOIN: all rows from both tables; unmatched sides are NULL.

Example:

```
SELECT a.id, b.val  
FROM A a  
LEFT JOIN B b ON a.id = b.id;
```

Tip: Explain with a quick Venn diagram in words; mention NULLs for missing matches.

2) How do you remove duplicate rows from a table?

Answer: Use ROW_NUMBER() over partition or GROUP BY/DISTINCT.

Common pattern: keep the first occurrence using window functions.

Example:

```
WITH cte AS (  
  SELECT *, ROW_NUMBER() OVER (PARTITION BY key_col ORDER BY  
    updated_at DESC) rn  
  FROM my_table  
)  
DELETE FROM my_table WHERE id IN (SELECT id FROM cte WHERE rn > 1);
```

Tip: Mention side-effects and to test with SELECT before DELETE.

3) How do you get the Nth highest salary (e.g., 2nd highest)?

Answer: Use ROW_NUMBER() or DENSE_RANK() depending on whether ties count.

Example:

```
SELECT salary FROM (  
  SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) rnk FROM  
  employees  
) t WHERE rnk = 2;
```

Tip: Explain tie behavior and when to use OFFSET...LIMIT alternative.

4) Explain window functions and give examples.

Answer: Window functions compute values across rows related to the current row without collapsing rows (e.g., ROW_NUMBER(), LEAD(), LAG(), SUM() OVER(...)).

Example: Running total:

```
SELECT dt, amt,  
  SUM(amt) OVER (ORDER BY dt ROWS BETWEEN UNBOUNDED PRECEDING  
  AND CURRENT ROW) running_total  
FROM sales;
```

Tip: Distinguish PARTITION BY vs ORDER BY in the OVER() clause.

5) What's a CTE and when to use it?

Answer: Common Table Expression (WITH clause) is a named temporary result set used for readability, recursion, or to break complex queries into steps.

Example:

```
WITH users_last_login AS (  
  SELECT user_id, MAX(login_ts) last_login FROM logins GROUP BY user_id  
)  
SELECT * FROM users_last_login WHERE last_login > '2025-01-01';
```

Tip: Use CTEs to document multi-step ETL logic; watch performance in some DBs (materialization differences).

6) How does EXPLAIN / query plan help with performance tuning?

Answer: EXPLAIN shows how the DB will execute a query (scan type, join order, index usage, cost). Use it to find full table scans and missing indexes or

inefficient joins.

Tip: Show a simple EXPLAIN and point out table scan vs index scan; mention ANALYZE for actual runtime when available.

7) What is an index? Types and trade-offs?

Answer: Indexes are data structures that speed lookups (B-tree, hash, bitmap). Trade-offs: faster reads, slower writes, storage overhead. Composite indexes help multi-column filters; order matters.

Tip: Mention index selectivity and avoid indexing low-cardinality columns unless needed.

8) Explain GROUP BY and HAVING vs WHERE.

Answer: WHERE filters rows before aggregation; HAVING filters groups after aggregation. GROUP BY collapses rows into groups for aggregate functions.

Example:

```
SELECT dept, COUNT(*) cnt
FROM emp
WHERE active = 1
GROUP BY dept
HAVING COUNT(*) > 10;
```

Tip: Mention you cannot use aggregate functions in WHERE.

9) How do you implement an upsert (insert or update)?

Answer: Use MERGE (ANSI), or DB-specific INSERT ... ON CONFLICT/ON DUPLICATE KEY UPDATE. For large datasets, use staging + MERGE.

Example (Postgres):

```
INSERT INTO t(id, val) VALUES (1, 'a')
ON CONFLICT (id) DO UPDATE SET val = EXCLUDED.val;
```

Tip: Talk about handling race conditions and transactional guarantees.

10) What is transaction isolation and ACID basics?

Answer: ACID: Atomicity, Consistency, Isolation, Durability. Isolation levels: READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ, SERIALIZABLE — trade off concurrency vs anomalies (dirty reads, non-repeatable reads, phantom reads).

Tip: Give an example when SERIALIZABLE is needed (strict correctness) and when READ COMMITTED suffices.

11) Explain partitioning and partition pruning.

Answer: Partitioning splits a large table by key/range (e.g., date) to improve performance and manageability. Partition pruning lets queries scan only relevant partitions (e.g., WHERE dt BETWEEN ...).

Tip: Use partition columns in WHERE to enable pruning; mention maintenance benefits (drop partition).

12) What are materialized views and when to use them?

Answer: Materialized views store precomputed query results for faster reads; must be refreshed. Use when heavy aggregations are read frequently and latency is acceptable.

Tip: Discuss refresh strategies (incremental vs full) and storage cost.

13) How do you handle slowly changing dimensions (SCD) in SQL?

Answer: Patterns:

- Type 1 – overwrite attributes.
- Type 2 – add new row with effective start/end dates (history).
- Type 3 – keep limited history in columns.

Tip: Show SCD Type 2 MERGE example and highlight surrogate keys & effective timestamps.

14) How to pivot/unpivot data in SQL?

Answer: Use CASE + GROUP BY for pivoting or DB-specific PIVOT/UNPIVOT features. Unpivot: transform columns to rows using UNION ALL or UNPIVOT.
Example (generic pivot):

```
SELECT id,  
       SUM(CASE WHEN metric='m1' THEN val END) m1,  
       SUM(CASE WHEN metric='m2' THEN val END) m2  
FROM facts GROUP BY id;
```

Tip: For many columns use automated ETL or generate SQL dynamically.

15) What's the difference between UNION and UNION ALL?

Answer: UNION removes duplicates (performs sort/dedup) while UNION ALL keeps all rows (faster). Use UNION ALL unless deduping is required.

Tip: Show performance implications on big datasets.

16) How do you perform a rolling window (e.g., last 7-day moving average)?

Answer: Use window frames with RANGE or ROWS in OVER() clause.

Example:

```
SELECT dt, avg(value) OVER (ORDER BY dt ROWS BETWEEN 6 PRECEDING  
AND CURRENT ROW) mov_avg  
FROM series;
```

Tip: Choose ROWS vs RANGE based on data distribution and ties.

17) How to detect and remove late-arriving or out-of-order events in event-time pipelines?

Answer: Use watermarking and event-time windows (often in streaming SQL engines). In batch SQL, deduplicate using ROW_NUMBER() by event_ts and ingestion_ts, keeping the latest ingestion for each event.

Tip: Explain ingestion_ts vs event_ts and mention allowed lateness logic.

18) What is a correlated subquery and when is it used? Performance concern?

Answer: A correlated subquery references outer query columns and executes per row — can be slow. Convert to JOIN or use window functions for performance.

Example:

```
SELECT e.*  
FROM emp e  
WHERE salary > (SELECT AVG(salary) FROM emp WHERE dept = e.dept);
```

Tip: Discuss rewrite opportunities using JOIN or WITH.

19) Explain normalization vs denormalization for analytical workloads.

Answer: Normalization reduces redundancy (OLTP) using many tables; denormalization stores pre-joined/aggregated data for faster analytical reads (OLAP). Data engineers often denormalize for query speed and simplified consumption.

Tip: Discuss trade-offs: update complexity, storage, and ETL cost.

20) How do you write efficient large-table joins (practical steps)?

Answer:

- Ensure join columns are indexed or partitioned appropriately.
 - Filter early (WHERE/ON predicates) to reduce row counts.
 - Use broadcast/small-table joins in distributed engines when one side is small.
 - Avoid functions on join columns (breaks index usage).
 - Prefer JOIN order / hints when optimizer misbehaves.
- Tip:** Mention checking EXPLAIN, statistics, and considering repartitioning or bucketing in distributed systems.

Quick practice tasks (do these to be interview-ready)

1. Write queries using ROW_NUMBER(), LAG(), LEAD() and SUM() OVER(...).
2. Show how MERGE or INSERT...ON CONFLICT works for upsert.
3. Run EXPLAIN on a slow query and explain the top 3 optimization steps.
4. Implement SCD Type 2 MERGE for a dimension table.